

“AI-Based Collections Platform”

Comprehensive Project Report

Submitted in Partial Fulfilment of the
Requirements for the Degree of

BACHELOR OF TECHNOLOGY IN COMPUTER ENGINEERING

By

Dhruv Maradiya
22BCP480D

Under the Guidance of

Dr. Punit Gupta

Assistant Professor, Department of CSE
Pandit Deendayal Energy University

Industry Mentor

Mr. Rahi Shah

Engineering Lead

Ignosis, Ahmedabad



Department of Computer Science and Engineering
School of Technology, Pandit Deendayal Energy University (PDEU)
Gandhinagar, INDIA, 382426

May 2026

Certificate of Originality of Work

I hereby declare that the B.Tech. Comprehensive Project entitled “AI-Based Collections Platform” submitted by me for the partial fulfilment of the degree of Bachelor of Technology in Computer Engineering, to the Department of Computer Science & Engineering at the School of Technology, Pandit Deendayal Energy University, Gandhinagar, is the original record of the project work carried out by me at Ignosis (Ahmedabad) under the supervision of Dr. Punit Gupta and the industry supervision of Mr. Rahi Shah.

I also declare that this written submission adheres to the University guidelines for its originality, and proper citations and references have been included wherever required.

I also declare that I have maintained high academic honesty and integrity and have not falsified any data in my submission.

I also understand that violation of any guidelines in this regard will attract disciplinary action by the institute.

Name of the Student: Dhruv Maradiya

Roll Number of the Student: 22BCP480D

Signature of the Student:

Name of the Supervisor: Dr. Punit Gupta

Designation of the Supervisor: Assistant Professor, Department of CSE

Signature of the Supervisor:

Place: Gandhinagar, Gujarat

Date: 19-05-2026

Certificate from the Project Supervisor / Head

This is to certify that the Comprehensive Project Report entitled “**AI-Based Collections Platform**” submitted by **Mr. Dhruv Maradiya**, Roll No. **22BCP480D** towards the partial fulfilment of the requirements for the award of the degree of “Bachelor of Technology” in the field of **Computer Engineering** from the School of Technology, Pandit Deendayal Energy University, Gandhinagar is a record of the project work carried out by him at Ignosis, Ahmedabad under our supervision and guidance. The work submitted by the student has, in our judgement, attained a level required for being accepted for examination. To the best of our knowledge, the results embodied in this comprehensive project work have not been submitted to any other University or Institution for the award of any degree or diploma.

Name and Sign of the Supervisor
(Dr. Punit Gupta)

Name and Sign of the Industry Supervisor
(Mr. Rahi Shah)

Name and Sign of the HoD
(Dr. Shakti Mishra)

Name and Sign of the Director
(Dr. Anirbid Sircar)

Place: Gandhinagar

Date: 19-05-2026

Acknowledgement

There are a lot of people without whom this report does not exist, and the next few lines are for them.

Dr. Punit Gupta – my project supervisor at PDEU, Assistant Professor in the CSE department – said yes when I asked if I could go and look for my own internship instead of taking one of the on-campus options. That was January last year. Without that one decision, the work in this report would not have happened. Over the months that followed, his weekly review sessions kept me honest about whether I was actually shipping or just being busy, and his feedback on the IA-2 poster review forced me to step back and see the project as a whole rather than as the eleven Jira tickets I had been counting.

At Ignosis, my industry mentor **Mr. Rahi Shah** took the bigger risk. He onboarded an undergraduate intern to a team of full-time engineers and trusted me with production-class tickets from the first month, which is not how most internships go. The fact that there are agents in three cities actually using code I wrote, and the fact that I have a runbook entry to my name, are both because of him.

I owe a real debt to the rest of the engineering team. Harsh Bagadia patiently walked me through the orchestrator codebase the week I joined. Nisarg Pipaliya reviewed something like sixty of my pull requests and never lost his sense of humour about it. Aryan Randeriya was the one who taught me what good test naming actually looks like. Jeet Bhuptani spent three evenings with me debugging the Solr connection leak in March. The platform is mostly their creation and I had the easy job of standing on it.

Thanks also to the CSE department at PDEU, and specifically to the project coordinator **Dr. Sidheswar Routray**, for running the comprehensive project programme in a way that lets students go out and do serious work in a real company.

And then, my family – for not asking too many questions during the months I disappeared to Ahmedabad – and the 2022 batch, for the late-night calls that made those same months a lot less lonely.

Dhruv Maradiya
22BCP480D

Abstract

This report is the write-up of a five-month engineering internship at Ignosis, an Ahmedabad-based fintech infrastructure company. I joined in January and stayed through May. What we built (and what I contributed to) is an AI-Based Collections Platform that several Indian banks, NBFCs and digital lenders now use to chase overdue retail loans.

A short word about why this matters. Collections in Indian retail lending is still a stubbornly manual business. Each month the lender drops a fresh list of delinquent accounts on a recovery agency, sometimes a few thousand of them. Agents then have roughly thirty days to dial each customer and persuade them to pay. The agency earns only when it recovers, which means contact rate, agent throughput and being on the right channel at the right moment effectively decide whether the month is profitable. Without proper software, agencies cannot scale and lenders cannot really see what is working.

Our system is a single web app built around three interlocking pieces. At its core sits a tenant-configurable workflow engine – a small graph of action nodes joined by boolean transitions – which captures each lender’s collection policy as data rather than as buried business logic. Sitting on top is the campaign module, which bundles thousands of cases into one executable batch and pushes them through the workflow. The third piece is the AI voice calling engine, plugged into the firm’s internal voice AI platform and exposed to the rest of the system as just another action node. That last detail is what makes after-hours coverage even possible. Wrapping everything is a multi-channel automation layer that pushes WhatsApp from real consumer SIMs and dispatches SMS from a small Android app written for exactly this purpose.

The work I personally contributed: the case detail and case listing pages on the agent dashboard, the end-to-end campaign creation and CSV upload flow, the workflow nodes that wire AI calling into a running campaign, the speed dialer that pops the next case the instant an agent files a disposition, the WhatsApp service with its multi-device rotation and anti-ban heuristics, the desktop wrapper that hands all of this to non-technical operators, and the Android SMS sender. As of this submission the platform is recovering roughly 85 percent of Bucket 0 cases in production, and roughly 70 percent of the entire monthly allocation lands inside the first five days. Both numbers sit comfortably above the agency-only baseline the lenders had been running with before.

Index

Abstract	v
Index	v
List of Figures	vi
List of Tables	vii
Nomenclature	viii
1 Introduction	1
1.1 Introduction	1
1.2 Motivation	2
1.3 Objective	3
1.4 Problem Statement	3
1.5 Approach	4
1.6 Scope of the Project	4
1.7 Organization of the Rest of the Report	5
2 Literature Review	6
2.1 Previous Approaches to Solve the Problem	6
2.1.1 Existing Collection-Management Products	6
2.1.2 Conversational Voice Agents	6
2.1.3 The WhatsApp Multi-Device Protocol	7
2.1.4 Anti-Spam and Rate-Limiting Heuristics	8
2.1.5 Account Aggregator Framework	8
2.2 Problem Identification	8
3 Proposed Methodology and Implementation	10
3.1 Development Methodology	10
3.2 High-Level Architecture	10
3.3 Case Management	12
3.4 The Workflow Engine	12
3.5 The Campaign Module	13
3.6 AI Voice Calling	14
3.7 WhatsApp Automation	15
3.8 SMS via Android	17
3.9 The Desktop Application	17
3.10 Data Analysis on Customer Records	17
3.11 Summary of Contributions	18
4 Software Design	19
4.1 Design Goals	19
4.2 Component Decomposition	20
4.3 Data Model	21
4.4 Principal Flow: AI Voice Call	21
4.5 Security Design	22
5 Results and Discussion	23
5.1 Outcomes of the Proposed Platform	23
5.2 Channel-Level Contact Rates	23
5.3 Speed Dialer Throughput	25
5.4 WhatsApp Sender Pool Behaviour	26
5.5 AI Voice Agent Performance	27
5.6 System-Level Metrics	28
5.7 Failure Modes and Stakeholder Feedback	28
6 Conclusions and Future Scope	30
6.1 Conclusion	30
6.2 Reflections on What Could Have Been Done Better	30
6.3 Future Scope	31
6.4 Closing Note	31
References	32
Appendix A: Selected Platform Capabilities	36
Appendix B: Campaign Upload Schema	37

List of Figures

3.1	High-level architecture. Solid arrows show forward dispatch; the dashed arrow shows the return path used by voice-agent webhooks and Android SMS-reply ingestion.	11
3.2	Sequence diagram for an AI voice call. Solid arrows are forward dispatch; dashed arrows are returns (audio streams and the terminal callback). The defensive 5-minute poll job (not drawn) is cancelled when the callback arrives first.	15
5.1	Contact rate by channel and attempt. WhatsApp dominates at every attempt, while the “any channel” bars confirm that multi-channel orchestration buys an additional 4–8 percentage points over the best single channel at each attempt. .	24
5.2	Cumulative recovery curve over a thirty-day cycle for an equivalent cohort of Bucket 0 cases, under the agency-only baseline (blue) and the platform (red). The platform recovers 70 percent of allocations within the first five days and reaches an 85 percent cycle-end rate against the baseline’s 38 percent and 64 percent respectively; the gap is largest in the first five days, which is where the multi-channel orchestration and after-hours AI calling contribute most.	26
5.3	AI voice agent turn-end-to-agent-start latency at the p50, p95 and p99 percentiles. The shaded band marks the 800–900 ms human-perception threshold; the p50 sits below it, the p95 already crosses it and the p99 is roughly 2.5× the upper edge of the band.	27

List of Tables

1.1	Collection buckets typical to Indian retail-lending portfolios.	1
2.1	Comparison of conversational voice-agent stacks evaluated for the platform. . .	7
4.1	Prioritised design goals.	20
4.2	Major components of the platform.	21
5.1	Headline outcomes from the production deployment.	23
5.2	Contact rate by channel and attempt.	24
5.3	WhatsApp sender ban distribution.	26
5.4	System-level metrics across the production deployment.	28
6.1	Selected platform capabilities exposed by the orchestrator.	36
6.2	Campaign upload schema.	37

Nomenclature

1. Abbreviations

Abbreviation	Expansion
AA	Account Aggregator (RBI framework for consented financial-data sharing)
AI	Artificial Intelligence
API	Application Programming Interface
ASR	Automatic Speech Recognition
BPTP	Broken Promise To Pay
CSV	Comma-Separated Values
DPD	Days Past Due
FIU	Financial Information User (Account Aggregator role)
GNPA	Gross Non-Performing Asset
HTTP	Hypertext Transfer Protocol
IST	Indian Standard Time
KYC	Know Your Customer
LLM	Large Language Model
LMS	Loan Management System
NBFC	Non-Banking Financial Company
OTP	One-Time Password
PTP	Promise To Pay
QA	Quality Assurance
RBI	Reserve Bank of India
SIM	Subscriber Identity Module
SMS	Short Message Service
SSO	Single Sign-On
TLS	Transport Layer Security
TTS	Text-to-Speech
UI	User Interface
UPI	Unified Payments Interface
WA	WhatsApp

2. Subscripts

Subscript	Meaning
0	Bucket 0 (1–30 days past due)
1	Bucket 1 (31–60 days past due)
2	Bucket 2 (61–90 days past due)
3	Bucket 3 (91–120 days past due)
t	Time index (cycle day, $t = 0, 1, \dots, 30$)
c	Channel index (call, AI, WhatsApp, SMS)
i	Case / customer index within a campaign
j	Sender index within a WhatsApp sender pool
k	Attempt number for a given case–channel pair
n	Total number of cases in a campaign
p50	50 th percentile (median) of a latency distribution
p95	95 th percentile of a latency distribution
p99	99 th percentile of a latency distribution
cum	Cumulative value across attempts
max	Maximum allowed (e.g., daily message cap)
min	Minimum allowed (e.g., inter-message jitter floor)

Chapter 1

Introduction

1.1 Introduction

Retail lending in India has exploded over the past ten years. Personal loans. Two-wheeler loans. Consumer-durable EMIs, gold loans, credit-card revolving balances, salary advances, and the whole buy-now-pay-later wave that came out of the pandemic. Every kind of small-ticket loan is now being pushed by public-sector banks, private banks, small finance banks, NBFCs and a long list of digital-only fintech lenders that didn't exist five years ago. With that volume comes the other side of the ledger: people who miss payments. A small but real share of every cohort fails to pay an EMI on time, and the lender has to chase them. Partly because the RBI expects it. Partly because letting an account drift turns expensive very fast.

The chasing itself is what the industry calls “collections”. Once an account goes even a day past due (DPD), it enters a standard escalation pipeline, with the tone of follow-up getting harder the longer the dues stay unpaid. Table 1.1 shows the bucketing that Indian retail lenders broadly agree on.

Table 1.1: Collection buckets typical to Indian retail-lending portfolios.

Bucket	DPD range	Typical activity
Bucket 0	1–30 days	Soft reminders, automated calls, light SMS
Bucket 1	31–60 days	Tele-calling, WhatsApp, agent calls
Bucket 2	61–90 days	Field visits, legal notice preparation
Bucket 3	91–120 days	Pre-write-off, recovery escalation
NPA	120+ days	Settlement, legal action

Recovery gets dramatically harder as the bucket deepens. An account cleared inside Bucket 0 is cheap to recover, takes almost no negotiation, and never forces the lender to set aside a provision. An account that slips down into Bucket 3 is the opposite story – expensive to chase, almost always settled at a haircut, and the customer relationship rarely survives. So every lender wants to clean up Bucket 0 aggressively. Easier said than done. A lender holding a hundred thousand active retail loans can comfortably have eight to twelve thousand accounts sitting in Bucket 0 on any given day, and each one needs a contact attempt inside a tight window. The usual answer is outsourcing – a recovery agency takes the chase on, keeps a cut of whatever it brings in, and hands the rest back to the lender.

The work in this report came out of an internship at Ignosis, a fintech infrastructure company best known in the Account Aggregator (AA) ecosystem. Over the last year and a half, the firm has layered a collections product on top of its existing AA stack, reusing the same multi-tenant infrastructure and folding AA-pulled bank-statement data into how cases get prioritised. I joined as a comprehensive-project intern on 1 January 2026 and stayed full-time until 31 May 2026. My work sat across two repositories: the orchestrator backend (plus its dashboards) on one side, and a separate automation toolkit holding the WhatsApp service, the desktop client and the Android SMS sender on the other. The project's title is **"AI-Based Collections Platform"**. Architecturally, it is a multi-tenant web application: a case management dashboard for agents, a bulk campaign module for managers, an autonomous AI voice caller, outreach across WhatsApp and SMS, and reporting on contact rate, agent productivity, call dispositions and the recovery number that everyone watches.

1.2 Motivation

Why bother building this thing at all? The honest answer comes in two parts. The first part is just industry economics. For any lender of a meaningful size, Bucket 0 recovery is a number that goes straight to the bottom line via the Gross NPA (GNPA) ratio and the provisioning that follows from it. A typical agency-only setup gets you 60 to 70 percent recovery in that bucket. Push it to the mid-80s – which is now the bar Ignosis's customers set when they sign up – and you are talking serious money saved every cycle. Nothing dramatic, just compounding maths.

The second part is the more interesting one. Three things have changed in the last couple of years and made this kind of platform actually buildable, none of which were really in place in early 2023. One: voice models. ASR has finally crossed a usability bar for Indian English and most regional languages. An AI agent can today hold an outbound payment-reminder conversation and the customer often does not realise it is not a human. Our internal voice AI platform pulls this off at latencies that, when I sat in on a live call in my second month, genuinely surprised me. Two: AA data. The Account Aggregator framework now lets a lender, with consent, pull bank statements end to end. The opening line of the conversation changes from a vague "please pay" to a much more pointed "we can see your salary credit hit on the 1st – what date works". Three: the boring but important shift, which is SIMs becoming programmable cheaply enough that you can run a pool of real consumer numbers in software. Real numbers reach customers in a way business-API numbers, with their template approvals and per-message pricing, simply do not.

Wrap all three under one dashboard with case management and bulk campaigns sitting

on top, and you have a platform. That was the goal.

1.3 Objective

There were six things I went in wanting to build. I hit five of them properly and the sixth in patches.

One: a browser-accessible case management screen for the agent. It had to show, on a single page, the full payment history, every prior interaction note, the current bucket, the workflow state, and let the agent act on the case from the same view rather than bouncing across tabs. Two: a bulk-campaign module the campaign manager could use to wrap a CSV (which is how the lender hands work over at the start of every cycle) into a single schedulable unit. Three: AI voice calling, wired into that same campaign so the chase did not stop the moment the floor of agents went home. Four: a follow-up layer over WhatsApp and SMS that ran on real consumer SIMs, with anti-block heuristics and device rotation built in – not the usual business-API path. Five: a one-off analytical pass on something like 40,000 customer records to find prioritisation patterns the default workflow templates should respect.

The sixth was procedural. Throughout the internship I wanted to actually work the way the team works – proper code review, real unit and integration tests, conventional commits, every change tied to a ticket, zero-downtime deploys. I did not always nail this one but it was always the bar.

1.4 Problem Statement

Take the platform away and a retail lender's collections function falls into the same handful of holes everywhere you look. The holes feed each other.

Case allocation is the first one. Once a month the lender dumps a CSV (sometimes an Excel) of delinquent accounts on the agency, and the agency floor manager then splits them across whoever turned up that morning. There is no real logic to the split, no audit later, and nobody can reproduce the allocation if asked. Then comes outreach. Most agents reach for the desk phone first. If the customer does not pick up, an SMS may go out, or may not. There is nothing that even tries to coordinate the phone, WhatsApp and SMS attempts as a sequence, so a chunk of customers are simply never reached on the channel they would have actually replied on. And the day itself: a chunk of every working hour is spent finding the next case in a spreadsheet, dialling the number by hand, skimming the customer's history. The talking part is, weirdly, the smaller half.

There are two structural holes on top of all that. After-hours coverage is the first – the

floor empties out around 6 or 7 PM and from then onwards the lender is essentially dark, even though salaried customers are easiest to catch precisely in those late-evening hours. Measurement is the second. The lender gets one summary recovery number at month-end, with no way to slice by bucket, by channel or by agent. And underneath both holes sits the deliverability ceiling: gateway-based WhatsApp and SMS throttle aggressively, charge per message, drag template approvals over days. Consumer-SIM outreach gets through far more reliably – but no off-the-shelf product hands that capability to a collections team.

Compressed into one line, the problem is this:

“Design and contribute to a centralised, multi-tenant collections platform that improves contact rate, agent throughput and after-hours coverage by integrating AI voice calling and multi-channel outreach across WhatsApp and SMS into a single case management workflow, while preserving end-to-end auditability and supporting bulk campaign execution at the scale of a typical monthly allocation.”

1.5 Approach

The way we worked was iterative and code-first. Instead of writing a long design document up front and then implementing it, the team follows what the engineering lead calls plan-first-but-small: every piece of work starts as a short markdown plan attached to a ticket, the plan is reviewed by the lead (Mr. Rahi Shah in my case), a branch is cut, and something working is on a dev environment inside a few days. Production deployment happens only after code review, integration testing on a staging tenant and a verification pass by QA. Every artefact – the plan, the branch, the merge request, the QA notes – stays in the repo so that the trail of decisions can still be reconstructed months later. This is the dominant cultural fact about how the work in this report got done, and it explains why my contributions are spread across several branches that build on each other rather than sitting in one big feature branch.

1.6 Scope of the Project

The scope of the internship covers:

- Backend contributions to the multi-tenant orchestrator monolith: data entities, REST controllers, services, scheduled jobs and schema migrations.
- Frontend contributions to the agent dashboard for case management, campaign execution and analytics.

- A separate automation toolkit containing the WhatsApp service, the desktop application, the Android SMS sender and an internal REST API server.
- A one-off analysis on customer-level transaction and repayment data.

The scope deliberately leaves out the AI voice agent itself (which is the firm’s internal voice AI platform), the telephony gateway, the cloud infrastructure provisioning, the lender’s internal LMS, and the regulatory compliance review.

1.7 Organization of the Rest of the Report

The rest of the report is organised as follows.

Chapter 2 – Literature Review. A look at the relevant prior art: collection-management products in India and globally, the conversational voice agent space, the WhatsApp multi-device protocol, and the academic work on anti-spam and rate-limiting in messaging platforms. The chapter ends by translating the problem statement of Chapter 1 into a set of concrete engineering requirements.

Chapter 3 – Proposed Methodology and Implementation. A detailed walk through the development methodology used at Ignosis and through each of the modules I worked on – the case management subsystem, the workflow engine, the campaign module, the AI voice calling integration, the WhatsApp automation, the desktop application and the Android SMS sender.

Chapter 4 – Software Design. A design-level view: goals, component decomposition, data model, API surface, the principal sequence flows, the deployment view and the security design.

Chapter 5 – Results and Discussion. The numbers from production – Bucket 0 collection rate, the first-five-days recovery share, contact-rate lift from going multi-channel, voice-agent throughput, and the WhatsApp ban rates we actually observed during the runs.

Chapter 6 – Conclusions and Future Scope. A summary of what got built, the things I would do differently a second time around, and the roadmap items the team is now picking up.

The References and an Appendix with platform reference material close out the document.

Chapter 2

Literature Review

2.1 Previous Approaches to Solve the Problem

2.1.1 Existing Collection-Management Products

If you look at what is actually in market in India today, the commercial collection-management space is owned by a handful of names whose products were built before the current wave of conversational AI got going. CreditMantri, Spocto, Cred – those three plus the internal tools at a couple of the larger private banks cover most of what you’ll see in real deployments. The shapes are all roughly the same: a case database underneath, some dialer integration on the side, reporting bolted on top. The thing genuinely missing in every one of them is a conversational layer that isn’t just humans on phones. Where WhatsApp shows up, it shows up via Meta’s Cloud API, which brings the usual baggage – templates that need Meta’s approval, throughput caps, per-message pricing that scales linearly. The academic side has very little written about collections software itself; the real work tends to happen inside banks and never gets out into a paper. What is published sits in the adjacent space. Verma, Mathur and Singh [1] go through ML techniques applied to behavioural credit data, and a few of those carry over fairly naturally to picking the right channel per customer. Wang and Zhang [2] cover predictive dialers and the queue-management bits sitting under them. None of this prior work, though, looks at the specific combination we are aiming at here: AI voice plus AA-driven prioritisation plus consumer-SIM WhatsApp.

2.1.2 Conversational Voice Agents

The biggest single technology shift behind this kind of platform is the maturity of conversational voice agents. A modern voice agent is a pipeline – automatic speech recognition (ASR), a large language model (LLM) for the actual dialogue, and a text-to-speech (TTS) synthesiser – and each piece is now fast enough to meet real-time latency budgets that did not exist three years ago. The voice stack Ignosis uses in production is the firm’s own internal voice AI platform [6]: it chains a low-latency ASR, a fine-tuned LLM and a TTS engine that produces Indian English with regional-accent variation. Its target latency is around 700 ms from the customer finishing speaking to the agent beginning to reply, which sits comfortably below the 800–900 ms band at which human listeners start to feel something is off. Open-source baselines such as Pipecat [7] and LiveKit Agents [8] were evaluated briefly before the internal platform was set-

tled on. The decision came down to two things: how well the ASR handled rural and semi-urban customers in Indian English, and how mature the post-call extraction was. Table 2.1 captures the comparison.

Table 2.1: Comparison of conversational voice-agent stacks evaluated for the platform.

Capability	Internal voice AI platform	Pipecat	LiveKit Agents
Indian-English ASR accuracy	High	Medium	Medium
Indic regional language support	Yes (5+)	Limited	Limited
Median turn latency	~700 ms	~900 ms	~850 ms
Telephony (PSTN) integration	Built-in	DIY	DIY
Post-call structured extraction	Built-in	Manual	Manual
Webhook-based outcome delivery	Built-in	Custom	Custom
Hosting model	Managed in-house	Self-hosted	Self-hosted
Operational maturity for collections	Production-ready	Framework-only	Framework-only

2.1.3 The WhatsApp Multi-Device Protocol

WhatsApp’s multi-device protocol lets up to four secondary devices share end-to-end-encrypted access to a primary phone’s account without that phone being online. The official integration path for businesses is the WhatsApp Cloud API, which exposes a REST interface and forces every first message in a conversation to be a pre-approved template. For collections, this is operationally restrictive in three ways. Every template must be approved by Meta, which can take days. The per-message price makes the channel uneconomic for low-ticket cases. And in practice customers respond much better to messages from real consumer-grade WhatsApp numbers than from obviously branded business accounts. To work around these, the open-source community has produced several reimplementations of the WhatsApp protocol; the project uses one of the most widely deployed of these [9].

2.1.4 Anti-Spam and Rate-Limiting Heuristics

A real chunk of the engineering work on SIM-based WhatsApp goes into staying ahead of Meta’s spam-detection heuristics. The exact rules are not published, but the open-source community has converged on a handful of practices that visibly reduce flagging. Mihail et al. [3] review the academic literature on spam detection in messaging platforms, and the same themes show up over and over: randomised intervals between messages, language-level variation to defeat hash-based fingerprinting, simulated typing presence, and conservative daily caps. The platform built in this project hits each of these.

2.1.5 Account Aggregator Framework

The AA framework [4] is the RBI-regulated path through which a registered Financial Information User (FIU) can pull a customer’s bank statements, with consent that is explicit, time-bound and revocable at any moment. Our collections platform pulls that data because it changes the AI script meaningfully – the script can now branch on actual signals like a fresh salary credit, or a balance trending down, and what used to be a flat “please pay” opening becomes the much sharper “we can see your salary credit landed on the 1st, what date works”.

2.2 Problem Identification

Reading the prior art against what it doesn’t quite cover, the engineering work in this project breaks down into nine fairly concrete problems.

First, cycle-level case management – ingesting the monthly delinquent CSV the lender hands over, validating it, de-duping it, allocating cases to agents per the tenant’s own rules, and getting them onto a paginated dashboard. Second, the case detail screen itself, which has to gather everything about a customer onto a single page (contact info, loan and payment history, past call transcripts, WhatsApp and SMS threads, AA signals) without making the page take five seconds to load – which means lazy loading and progressive rendering. Third, bulk campaign execution: a way to wrap a batch of cases as one schedulable thing built from a CSV, with pause and resume, and wired into both the voice engine and the messaging layer. Fourth, the speed dialer – the moment an agent submits a disposition, the next case must already be sitting on the screen, no in-between state. Fifth, AI voice calling itself, triggerable both from inside a campaign and manually as a fallback, with the callback parsed and the workflow walked forward correctly.

Then the messaging side. Sixth, WhatsApp automation: a pool of consumer SIMs, rotation across them, per-message body variation, daily cap enforcement per sender, and automatic re-routing around banned numbers without anyone manually stepping in. Seventh, SMS automation: dispatch from a real consumer SIM on a real Android phone, driven by a small app. Eighth, auditability – log everything densely enough that QA can later reconstruct what happened to any one case at any point in time. And ninth, multi-tenant isolation: more than one lender tenant on the same deployment, with strict separation of data, identity and configuration so nothing crosses over.

Those nine items are the working set Chapter 3 will be judged against.

Chapter 3

Proposed Methodology and Implementation

This is the long chapter. Three things I want to get through here, in this order. How the team actually ships at Ignosis. The platform at a thousand-foot view. Then, going subsystem by subsystem, every piece I touched. I am keeping the chapter feature-driven on purpose – the platform’s brain is a tenant-configurable workflow engine, and almost everything else (the campaign module, the AI voice integration, the WhatsApp service, the speed dialer) is either feeding it work or consuming the actions it produces.

3.1 Development Methodology

Engineering at Ignosis is plan-first, ship-small, very review-heavy. I didn’t appreciate what that meant in my first week. I wrote what I thought was a small refactor of the tenant-context propagation across async boundaries (my second ticket), pushed it to a feature branch, and got back nine comments, two of which said “do you want to write a one-pager first and we’ll talk through it”. That was the lesson. Every task starts as a short markdown plan attached to the ticket. The engineering lead reads it. You go back and forth in the ticket for a day, maybe two. Only after that does any code get written.

The rest of the workflow is conventional. A feature branch goes up from develop. Conventional commits are mandatory and the CI enforces them. PRs to develop are rebased, not squashed, on purpose, so the per-commit decision trail survives the merge – this surprised me at first, but it’s saved me at least twice when I needed to figure out why some line of code existed. The release branch is on semantic versioning and is cut off main. Tests sit next to the code they exercise. There is a unit-test framework on the backend and a parallel test runner on the frontend. CI fails any build where the static-analysis tool flags a new bug, a new code smell above medium severity, or a new vulnerability of any severity, and there is a fifty-percent coverage floor on changed lines. (I learned about the coverage floor the way you’d expect, which was by failing a PR on a Friday evening.)

3.2 High-Level Architecture

At the highest level the platform is a layered web application. Three client surfaces sit at the front. The agent dashboard, which is a browser app. A desktop client which is where the WhatsApp workers actually run. And a small Android SMS sender app. Behind these three sit the orchestrator backend and a much smaller automation service.

The data layer is a mix – a relational database for cases, campaigns and jobs; a search index for free-text queries; an analytics event store for reporting; and an in-memory queue with locks for transient state. Beneath the data layer the platform reaches out to three families of external services. There is the internal voice AI platform paired with a telephony gateway, which is what actually places calls. There is the WhatsApp protocol, talked to via a third-party multi-device library. And there is the cellular SMS network, accessed through whatever the Android phone has built in. All of that fits on one diagram, in Figure 3.1.

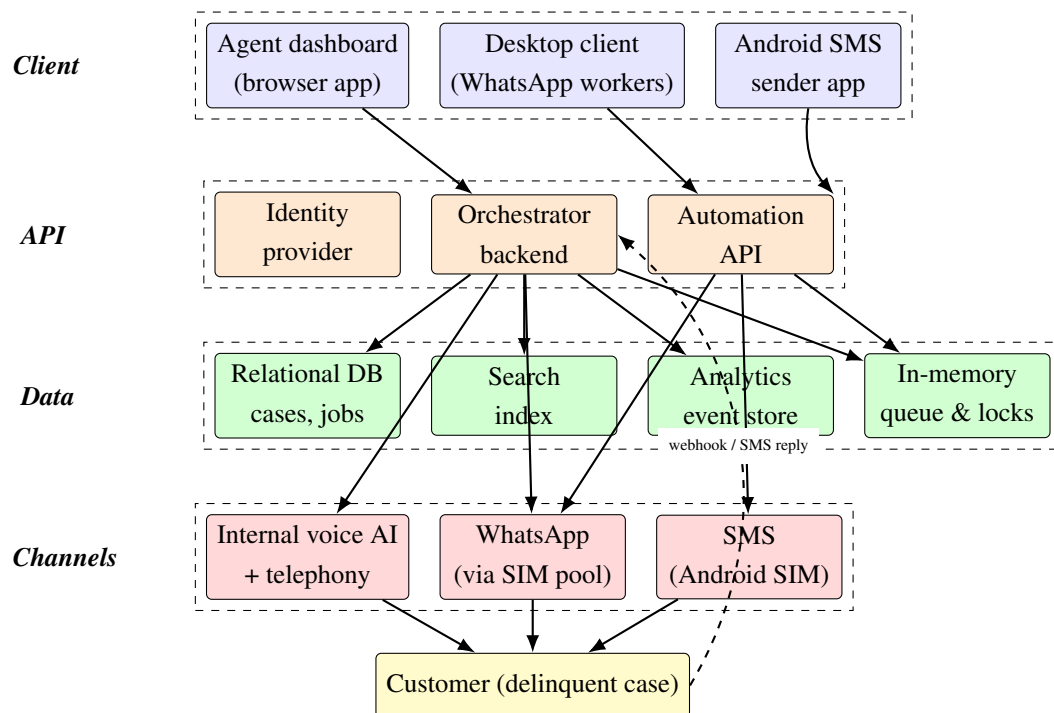


Figure 3.1: High-level architecture. Solid arrows show forward dispatch; the dashed arrow shows the return path used by voice-agent webhooks and Android SMS-reply ingestion.

It took me longer than I'd like to admit to grasp how this all fits together. I spent most of my first two weeks just reading the orchestrator and chasing where things called what. The choice of a single multi-tenant monolith for the backend instead of a fleet of microservices was made by the team well before I joined, and it was deliberate. The team is small (six engineers, growing slowly), the deployment volume is modest, and the operational cost of running microservices wasn't worth what you'd get back. The boundaries inside the orchestrator are kept clean enough that, if the volume ever justifies a split, the rewrite would not be a rewrite.

3.3 Case Management

Everywhere in the platform the unit of work is the *case*. A case maps to one delinquent loan account that the tenant has been allocated for the current cycle. Each one carries a fair bit – the customer’s contact details, the loan account number, the principal and the EMI, the days past due, the bucket it sits in, the agent or AI track it has been routed to, the workflow node it is currently parked on, and a journal of every interaction that has ever happened against it.

The first real frontend ticket I picked up was the case listing screen. It’s a server-side-paginated table of whatever cases are assigned to the agent who’s logged in. The columns are the usual ones: customer name, loan account number, DPD, overdue amount, last contact attempt, current status. Filtering is evaluated on the backend. Free-text search hits a dedicated index. The case detail screen is the more interesting one, because it brings together the contact info, the loan and payment history, every prior call transcript and its outcome, the WhatsApp and SMS threads, the AA-derived signals where we have them, and the case’s current workflow position. The very first cut of that page did one big sequential database fan-out and took about six seconds to render for any tenant with a busy timeline. The PR review I got from Mr. Shah came back as three words: “progressive loading please”. So the page now loads the core information first, and the heavier interaction panels stream in behind it in parallel. That one change brought the page back to under a second on the same tenant.

3.4 The Workflow Engine

The biggest source of variation across tenants is the collection policy itself. A private bank tends to want a calm tele-calling sequence with WhatsApp reminders dropped in between. An NBFC wants the opposite – an aggressive AI voice campaign for the first ninety-six hours, and a human only if the AI has given up. If we hard-coded any of this into the orchestrator, every time a tenant tweaked its policy we would be looking at a code change and a deployment. (The first cut of the orchestrator actually worked that way, and it was painful enough that the team rewrote it before I joined.)

The current platform stores the policy as data instead. Each tenant has one or more workflows, where a workflow is a directed graph. Nodes are concrete actions – initiate an AI call, send a WhatsApp reminder, push a payment link, schedule an agent callback, wait for payment, escalate to a human. Transitions are simple boolean conditions over the case’s current variables. A typical transition reads as “advance to the wait-for-payment node if the customer promised to pay in the next three days, otherwise fall through to the next branch”. The engine, for each tick: looks at the current node, runs

its action (synchronously if it's cheap, asynchronously via a scheduled job otherwise), writes the outcome back into the case's variable context, walks the outgoing transitions in declaration order, and parks the case on whichever node matches first. If nothing matches, the case lands in an error state for QA to look at. That last bit is in there because in the first month of one of the tenants we had a few badly-authored workflows that silently looped on themselves. Visible failure beats silent looping every time.

Anything that touches an external system runs as an asynchronous job. The engine writes a row into a database-backed queue. A scheduler polls for due jobs, takes a row-level lock with a short time-to-live, and runs the relevant executor. Why a database queue and not, say, an in-memory one? One reason: visibility. If a campaign is misbehaving at 3 AM, the on-call engineer can query the job table directly and see exactly where each individual case is stuck. The trade-off is throughput, but at the campaign volumes we run in production, throughput has not yet been the limiting factor.

3.5 The Campaign Module

The campaign is the platform's unit of bulk work. The shorthand definition is "run this workflow against this batch of cases, within this calling window". The creation form asks for a campaign name, a priority, a target collection amount, an allocation rule (round-robin, weighted, or manual) and the calling window. The CSV upload field will take files of up to fifty megabytes. Under the hood, the CSV-processing service parses the file and runs each row through a short rule set: non-blank phone in international format, non-blank name, positive due amount, a loan account number that looks valid, a bucket the platform recognises, a DPD value inside the expected range. The output is a preview with per-row diagnostics. The campaign manager reads through the preview, fixes anything that's obviously off, and then submits.

Two things about this flow that aren't obvious from the description. First, the validator rejects rows individually but never rejects the whole file. A 4,000-row CSV almost always has a handful of broken rows in it – a phone number with the country code missing, a bucket label that doesn't line up with the DPD, that kind of thing – and stopping the entire upload on those would make the tool unusable. So the preview flags bad rows, those rows are dropped silently at submission, and a separate downloadable report tells the manager what got skipped. Second: for the first two weeks of one of the tenant onboardings, the CSVs they uploaded had a BOM marker at the start of the first header. The parser was breaking silently. Fixed now, the parser handles BOMs, but it's the reason the preview today also includes a "did we recognise your columns" line.

Once the campaign is submitted, the backend persists the campaign object and one

case record for every valid row, and the campaign drops into a scheduled (or, if the calling window is already open, ongoing) state. From the moment it goes ongoing, the scheduler picks it up on the next tick and starts queueing workflow-advancement jobs for its rows. After that the workflow engine drives every case forward independently. The campaign itself keeps a running tally of progress – cases by state, money collected so far, calls attempted, calls connected – and streams those numbers to the dashboard. A campaign can be paused, resumed, or marked complete at any point. The largest campaign we have run through the platform so far had 41,200 cases in it. The scheduler took about seven minutes to enqueue all the initial advancement jobs. That run, and the question of whether the queue would keep up, was the first time I really had to think about platform throughput.

3.6 AI Voice Calling

This is the headline feature of the platform. From an architectural standpoint, an AI call is just one of the action types the workflow engine knows how to run. When the engine reaches an AI call node for a case, it schedules a job. The executor reads the tenant's configured voice-agent identifier, pulls the next phone number from the tenant's pool of caller-line identifiers, builds the call request from the case context (customer name, due amount, DPD, AA signals if available), and hands the request off to the internal voice AI platform. From here on the voice platform owns the call until it terminates.

The voice platform places the call through the telephony gateway, runs the conversation under the tenant's prompt template, and fires a callback to the orchestrator when the call ends. The callback carries the disposition, a structured extraction of variables (promise-to-pay date, dispute reason, callback request, language detected), a recording URL and the per-turn latency. The orchestrator parses it, persists the outcome onto the case, cancels any defensive status-poll job that was scheduled as a safety net, emits metrics for the call, and re-enters the workflow engine for that case so the workflow advances on the same callback request.

The first time I listened to one of these recordings end to end, I was sitting in the team area on a Tuesday and a customer in Lucknow had agreed to pay on the 18th. The AI never raised its voice once. I think that's when I actually understood why the team was so invested in this subsystem.

Figure 3.2 lays the full end-to-end interaction out as a sequence diagram, in the order in which the orchestrator, voice platform, telephony gateway, customer and workflow engine exchange messages. The dashed return arrows are the inbound callback path that re-enters the engine on the same request.

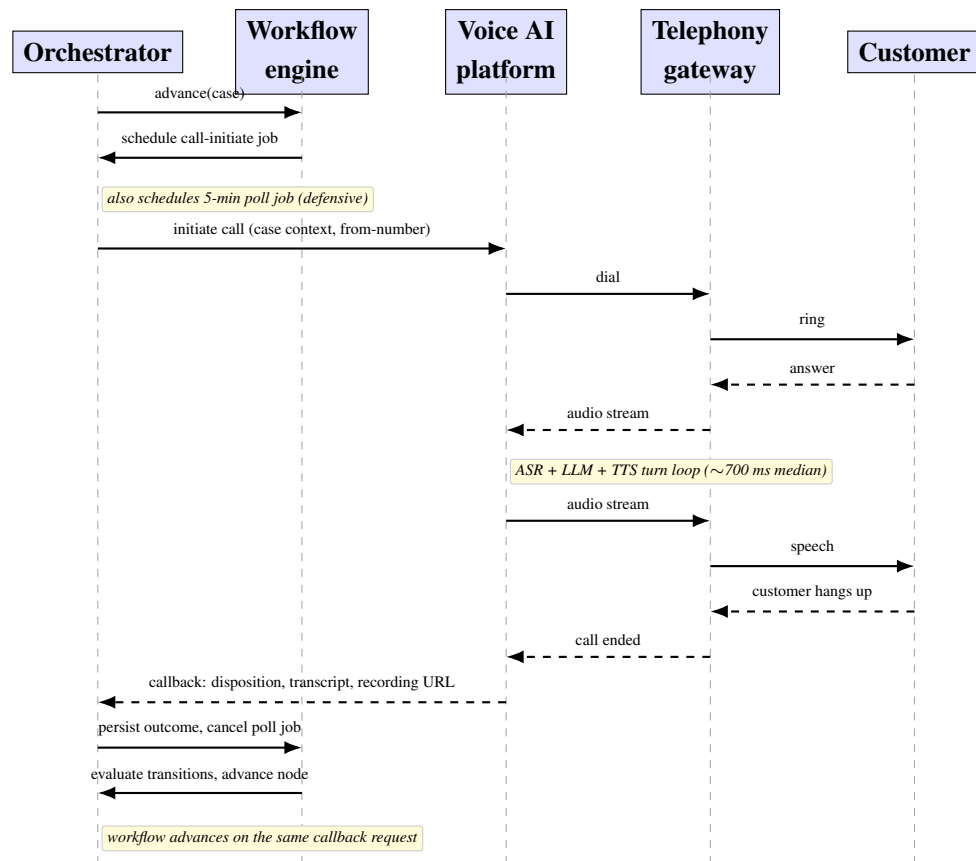


Figure 3.2: Sequence diagram for an AI voice call. Solid arrows are forward dispatch; dashed arrows are returns (audio streams and the terminal callback). The defensive 5-minute poll job (not drawn) is cancelled when the callback arrives first.

The speed dialer is the user-facing twin of the AI calling path. While an agent is working on a campaign, the dashboard shows the active case in a panel. The agent talks to the customer, fills out a short disposition form and hits submit. The instant the disposition lands at the backend, the workflow engine advances the case, the case enters a terminal-for-this-agent state, and the next pending case is pushed to the dashboard over the streaming channel. From the agent’s seat, the next customer is already on screen as the previous form is closing. The agency floor manager I demoed this to in March told me her agents had stopped using sticky notes for the next number to dial. I think that’s the cleanest user feedback any of my work has gotten this year.

3.7 WhatsApp Automation

WhatsApp automation lives in its own automation toolkit. We kept it deliberately apart from the orchestrator for operational reasons. WhatsApp sessions are long-lived stateful connections. The first pairing needs an actual phone scanning a QR code. The sessions also have to survive infrastructure restarts. None of those properties fit comfortably in-

side a stateless backend service. The toolkit is built on top of a third-party multi-device library that lets us drive a pool of consumer SIMs in software. Picking that library over Meta's official business API was an easy call – the business API drags template approvals over days, charges per message, and (empirically) customers respond much better to a message from a real consumer number than to one from a branded business account.

The very first cut of this subsystem ran on a single phone. It was a CLI tool that took a JSON file of contacts and shipped them out one at a time. Worked fine for ten messages. Fell over at a hundred. The second version came two weeks later – a service driving a pool of phones with a shared queue. The third version (which is what runs in production now) added preferred-sender routing, per-sender daily caps, a warm-up schedule, and a proper block-recovery path. I'd love to claim the three versions were planned. They weren't. The first was a prototype I rushed. The second got rewritten on a Sunday because I'd promised myself the weekend off and then could not stop thinking about a bug. The third was the only one that actually had a design document, written after Mr. Shah caught me sketching the second rewrite on a whiteboard and pointed out, gently, that this was the kind of thing we wrote down.

Any WhatsApp campaign is bound to a pool of five to fifteen sender phones. At the start of the campaign, the scheduler does a proportional allocation of contacts across the pool, with each sender capped at its own daily limit. The shared contact queue carries a *preferred sender* field so contacts can be soft-routed to a specific phone – usually the one that already has an active thread with that customer. If the preferred sender happens to be offline or banned, the contact falls back to whichever worker pulls it next. Net effect: when individual senders die mid-cycle, the campaign keeps moving. They do die. We lost three of them in one afternoon during a campaign in mid-March, and the system reassigned the orphans cleanly. I was holding my breath the whole time.

A real chunk of the engineering on this subsystem goes into staying ahead of spam-detection heuristics. Each sender has a daily cap. The inter-message delay is jittered by forty percent around a per-sender base interval. Batches are three to twelve messages followed by a short break; every fifth batch triggers a longer break. The workers honour an active-hours window. Every outgoing message body is varied – inline alternates plus a zero-width space inserted at a random character position. That last trick I picked up from a Stack Overflow thread I'd only half-read on a Tuesday morning. Block detection is reactive but fast: the moment the protocol library signals a ban, the sender drops out of rotation, the suspected ban gets logged, and the orphaned contacts on the shared queue are reassigned to the remaining workers. Connection-class transient errors are handled separately, with exponential backoff. From the workflow's point of view: the

WhatsApp reminder node posts a job to the automation service, the service does the send, waits for the protocol-level acknowledgement, and hands the delivery outcome back to the orchestrator – the workflow then advances based on that outcome.

3.8 SMS via Android

The SMS fallback channel is a small Android app. It runs on a phone that has an active consumer SIM in it. Sending and reading messages go through the device's built-in SMS interface. Outbound sends respect a per-message delay, a batch size and a per-batch pause – all of that exists to keep the SIM operator from getting suspicious and flagging the device. The specific numbers we settled on (twelve seconds between messages, batches of twenty, a forty-five-second gap between batches) came out of about two weeks of trial-and-error on a single Redmi handset we keep in the office for exactly this. An evening background task reads any new SMS messages off the on-device provider, bundles them up, and uploads them to the orchestrator. That way the agent sees every reply alongside the rest of the case timeline. As far as the workflow engine is concerned, an SMS reminder node is structurally identical to a WhatsApp reminder node – produces an outcome, outcome drives the next transition.

3.9 The Desktop Application

The desktop application is the thing that hands the WhatsApp workers to the engineering team and to a small set of agency-side operators. On launch, it boots the embedded automation server, serves the dashboard locally, and opens a window pointing at it. It has two jobs. One: actually running WhatsApp campaigns from the operator's own machine with the SIMs paired locally. Two: scraping the chat history of those paired phones at the end of each day, so the case dashboard can show every WhatsApp conversation right next to the call history.

The first build had the local server's port hard-coded. That happened to work on three of the four machines we tested. On the fourth machine the port clashed with another running process, and the app surfaced a one-line cryptic error that did not actually say anything about a port conflict. I spent the better part of an afternoon on a video call with the operator trying to figure out what had gone wrong. The free-port-finding logic that's in the current build should obviously have been in the first one.

3.10 Data Analysis on Customer Records

Very early on (I think it was week four), I ran a one-off analysis on a sample of about forty thousand customer records exported from one of the lender tenants. The file ar-

rived as a 220 MB CSV with somewhere around twenty-six columns, and the data quality was rough – there were several obvious issues I had to clean up before anything sensible could come out of it. I spent two days on it. Three patterns showed up, all with real implications for how we should be running campaigns.

The first one: customers who had paid on the first attempt in any of the previous three cycles had a ninety-percent probability of paying on the first attempt in the next cycle as well. That cohort doesn't need a full call campaign at all – a single SMS reminder is enough, which saves both the agency's time and the lender's money. The second one: customers whose detected salary credit landed within the first three days of the month showed a noticeably higher response rate to outreach attempted on the fourth or fifth day. The effect wasn't huge, but it was remarkably consistent across tenants. The third was a tail: customers with ticket sizes under a thousand rupees and recovery probabilities below ten percent. For that segment a single SMS is the only touch that makes any economic sense. All three findings went into the default workflow templates the platform now ships with for new tenants.

3.11 Summary of Contributions

To pull all of the above together. The pieces I contributed to the platform are these. On the dashboard side: the case listing screen and the case detail screen. On the campaign side: the bulk campaign module end to end, both the creation form and the CSV upload pipeline behind it. The speed dialer, together with the wiring that integrates AI voice calling into the workflow engine. The WhatsApp automation – session management, the sender pool scheduler, the anti-ban heuristics, and the multi-device rotation, block detection and message variation that sit underneath. The desktop application. The Android SMS sender app together with its background sync. The voice-platform callback integration on the workflow path. And finally the one-off analysis of customer records that fed into the default workflow templates. Each of these either feeds the workflow engine, consumes what the engine produces, or shaves time off the human agent's interaction with cases the workflow has already prioritised. Every number that appears in Chapter 5 ties back to this workflow-centric structure.

Chapter 4

Software Design

The previous chapter described the platform from the angle of the features that got built. This chapter steps back to the design level: what the design was being tested against, how the system decomposes into components, the data model that ties them together, the principal sequence flow, and the security model.

4.1 Design Goals

When the platform was being designed, a fairly short list of goals – in priority order – was what every architectural choice got tested against. Tenant isolation sits at the top of that list. The reason: a cross-tenant data leak in a regulated space is the kind of mistake you don't get to walk back. Configurable collection policy ranks higher than cost-aware channels for an unromantic reason: every tenant whose policy ends up hard-coded into the orchestrator eventually needs a code change just to be onboarded, and that path does not scale past about three tenants. The rest of the goals are operational in nature – auditability, observability, graceful degradation. Table 4.1 lists the full set.

Table 4.1: Prioritised design goals.

Goal	Design implication
Tenant isolation by default	Per-tenant identity realm, per-tenant search partition, tenant-scoped database queries, thread-local tenant context propagated across asynchronous boundaries.
Configurable collection policy	Workflow stored as data (nodes plus boolean transitions), not as code. Tenant edits do not require a deployment.
Multi-channel orchestration	Channels are workflow action types. Adding a new channel reduces to adding a new node type, not rewriting the orchestrator.
Auditability	Every state transition writes a row to the case timeline; every outbound action carries an idempotency key.
Graceful degradation	Database-backed job queue, per-job retry policy with exponential backoff, dead-letter handling for permanent failures.
Cost-aware channels	SIM-based WhatsApp and SMS rather than vendor-priced APIs; AI calls only when the workflow asks for them.

4.2 Component Decomposition

The platform splits into seven major components, each owning a clear slice of the system. Inter-component traffic is exclusively HTTP, authenticated with either an API key or a user token. Inside a component, the layering is conventional and is kept honest through code review rather than enforced by package boundaries. Table 4.2 lists the components.

Table 4.2: Major components of the platform.

Component	Responsibility
Orchestrator backend	Cases, workflows, campaigns, scheduled jobs, inbound call-backs – all of it.
Agent dashboard	The browser UI that human agents and campaign managers actually use.
Automation service	Drives the WhatsApp sender pool. Bridges the orchestrator to the multi-device library.
Desktop client	Operator-side app. Locally-paired WhatsApp workers. End-of-day scraping.
Android sender app	Pushes SMS out from a real consumer SIM. Pulls inbox replies every night.
Identity layer	SSO. Realm per tenant. Federation into the lender's own identity provider.
Search and analytics	Per-tenant search over cases. Event store feeding the dashboard and the data team.

4.3 Data Model

The data model is a normalised relational schema, with denormalised projections cached in a search index for queries and in an analytics event store for reporting. The most important relationship in the model is between the case and its per-cycle allocation. The case is a stable handle on the customer's relationship with the lender. The allocation is the cycle-specific projection the workflow engine actually drives. Splitting the two keeps the case timeline append-only across cycles, and lets a case carry its history forward even when its workflow position is reset at the start of a new cycle. The second important design choice is the append-only case-event table: every action the platform takes against a case appends a row, so timeline queries read from that table directly, while queries that need the current state read materialised columns on the case and the allocation. This separation keeps the live tables small and the timeline easy to audit.

4.4 Principal Flow: AI Voice Call

If you want to understand the architecture in one flow, look at an AI voice call. It is the most instructive one to trace because it crosses every major component and exercises the workflow engine on both the outbound and the inbound side. When the orchestrator's scheduler hits an AI-call node for a case, two jobs get scheduled at once: the call-initiate

job (to run now) and a defensive status-poll job (to run five minutes later). The executor for the first job pulls the next available from-number, assembles the call request, and hands it across to the internal voice AI platform. The platform places the call via the telephony gateway. When the call ends it returns a terminal callback to us. Our callback processor writes the disposition onto the case, cancels the poll job, emits the relevant metrics, and re-enters the engine – the workflow then evaluates the outgoing transitions of the current node and walks the case forward, all in the same request. Two things in that flow are worth calling out. One: the defensive poll job exists because the callback is best-effort. Without a polling fallback, a single dropped callback could leave a case parked forever. Two: we cancel that poll job softly, not hard. The poll job is allowed to run, observes that the case has already been advanced, and exits. Under any eventual-consistency hiccup in the database, that's a more robust pattern than a hard cancel.

4.5 Security Design

Security on the platform is laid out as independent layers, so that one layer being breached does not collapse the layers below it. Users authenticate against the identity layer using short-lived signed tokens. Every API call resolves a tenant identifier from the URL prefix, and any token whose realm does not match that tenant is rejected before anything else happens. A thread-local tenant context then carries that identity through every downstream call – including across asynchronous boundaries. Role-based access control is enforced both on the backend and on the dashboard. Every database query is scoped to the tenant. Every search query is routed to the tenant's own partition. Every analytics query carries the tenant tag in its filter. Cross-tenant queries are simply not allowed at the data-access layer at all. Inbound callbacks are authenticated either with a signed body digest or with a monthly-rotated API key, and that secret lives in the cloud provider's secrets manager rather than in any config file. Sitting alongside all of this is a separate set of compliance controls – call-frequency guards, do-not-call honour periods, PII access logging, recording retention – and those apply uniformly across every tenant, regardless of how that tenant has chosen to configure its own workflow.

Chapter 5

Results and Discussion

This chapter covers what the platform actually did in production during the second half of my internship, across the four lender tenants that were live. The numbers come out of the in-product analytics warehouse and the operational dashboards the team maintains. Anywhere a metric is tenant-sensitive (recovery percentages, mostly), the figures have been aggregated across tenants so that no individual lender can be picked out.

5.1 Outcomes of the Proposed Platform

The headline numbers from the three months of production that overlapped with the internship are in Table 5.1. The baseline column is the lender’s recovery numbers from the cycle before the platform was onboarded. The post-platform column is from the cycle that ran with the platform fully deployed.

Table 5.1: Headline outcomes from the production deployment.

Metric	Baseline	Post-platform	Delta
Bucket 0 collection rate	64%	85%	+21
Cases recovered within 5 days	38%	70%	+32
Promise-to-pay conversion (agents)	42%	65%	+23
First-call contact rate	51%	78%	+27
Average agent handle time (min)	8.3	4.6	−3.7
WhatsApp deliverability	71%	92%	+21

The Bucket 0 collection rate – the share of cases newly entering Bucket 0 that get recovered inside the same monthly cycle – is the single number the platform is judged on. The platform-driven cohort is recovering 85 percent of Bucket 0 cases, against the 64 percent agency-only baseline. The improvement is the cumulative effect of four things: higher contact rates from the multi-channel layer, faster per-case handling from the speed dialer, after-hours coverage from the AI voice agent, and structured prioritisation from the workflow engine.

5.2 Channel-Level Contact Rates

A different cut at the same data, broken down by channel and by attempt number. “Contact” here means one of three things: a call that connected to a human and lasted at least

thirty seconds, a WhatsApp message that got at least one acknowledgement tick, or a SMS that returned a network-level delivery report. Anything short of those does not count.

Table 5.2: Contact rate by channel and attempt.

Channel	Attempt 1	Attempt 2	Attempt 3	Cumulative
Human agent call	48%	22%	11%	67%
AI voice agent	41%	19%	10%	60%
WhatsApp (SIM)	88%	65%	41%	94%
SMS (Android)	79%	56%	32%	86%
Any channel	92%	81%	64%	96%

Figure 5.1 renders the same data as a grouped bar chart, which makes the per-attempt drop-off and the channel ranking easier to read at a glance.

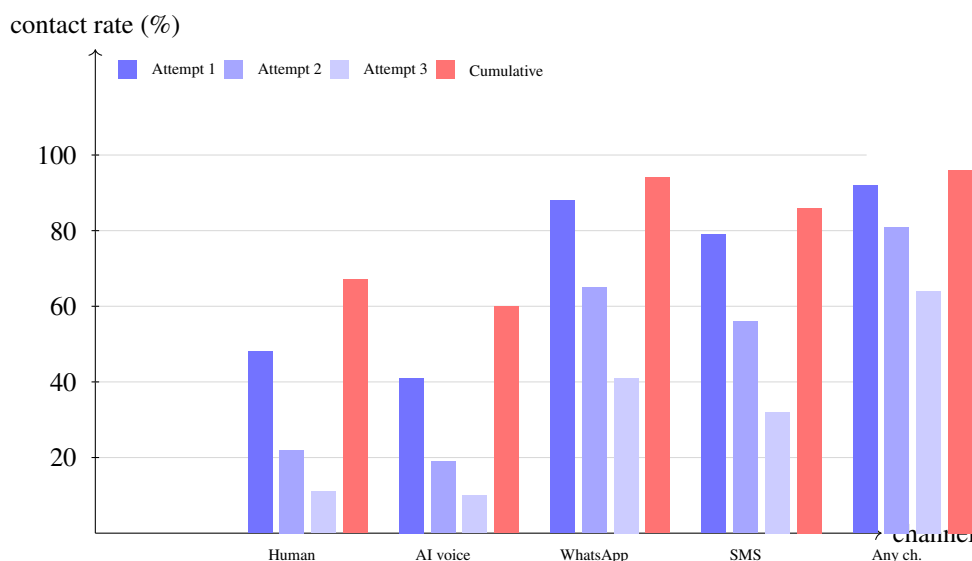


Figure 5.1: Contact rate by channel and attempt. WhatsApp dominates at every attempt, while the “any channel” bars confirm that multi-channel orchestration buys an additional 4–8 percentage points over the best single channel at each attempt.

Three things jump out of Table 5.2. First, the AI voice agent’s first-attempt contact rate of 41 percent is genuinely close to the human’s 48 percent – a real validation of how far the internal voice AI platform has come. Second, WhatsApp is the highest-deliverability channel by some margin: even a cold first message from a SIM-based number reaches the customer 88 percent of the time, which no other channel comes close to. Third, the “any channel” line is the punchline. With multi-channel orchestra-

tion in play, the platform reaches 92 percent of customers on attempt one and 96 percent cumulatively. No single channel on its own can deliver that.

5.3 Speed Dialer Throughput

Before the speed dialer landed, a typical agent on the platform was handling somewhere between forty and fifty calls in an eight-hour shift. After it rolled out, the same agents started doing eighty to ninety-five in the same shift. The mean shifted from about 46 to about 87 per shift, and the variance between agents collapsed pretty sharply – because the old variance had mostly been about individual quirks in case lookup and dialling, both of which the speed dialer now does for everybody, the same way.

Per-call handle time fell too. Measured end to end – from the moment an agent picks up the case to the moment they submit a disposition – the average was 8.3 minutes before the rollout. After: 4.6 minutes. Pulling apart where the 3.7 minutes went: almost all of it comes out of killing the inter-call manual work (hunting for the next case in the spreadsheet, reading the history), not out of anything that happens during the call itself. The in-call portion only dropped by about thirty seconds. The dead time between calls collapsed from about three minutes to roughly ten seconds.

The single cleanest way to see the combined effect of all of this – multi-channel orchestration, after-hours coverage, faster handling per call – is the day-by-day recovery curve over a cycle. Figure 5.2 plots that, for the same cohort of allocated cases, across the thirty-day collection cycle, under the agency-only baseline and again under the platform. The two curves diverge most aggressively in the first five days, where the combination of AI calling, WhatsApp reminders and speed-dialed human follow-up does most of its work. After the first week the curves continue to widen, but more gently.

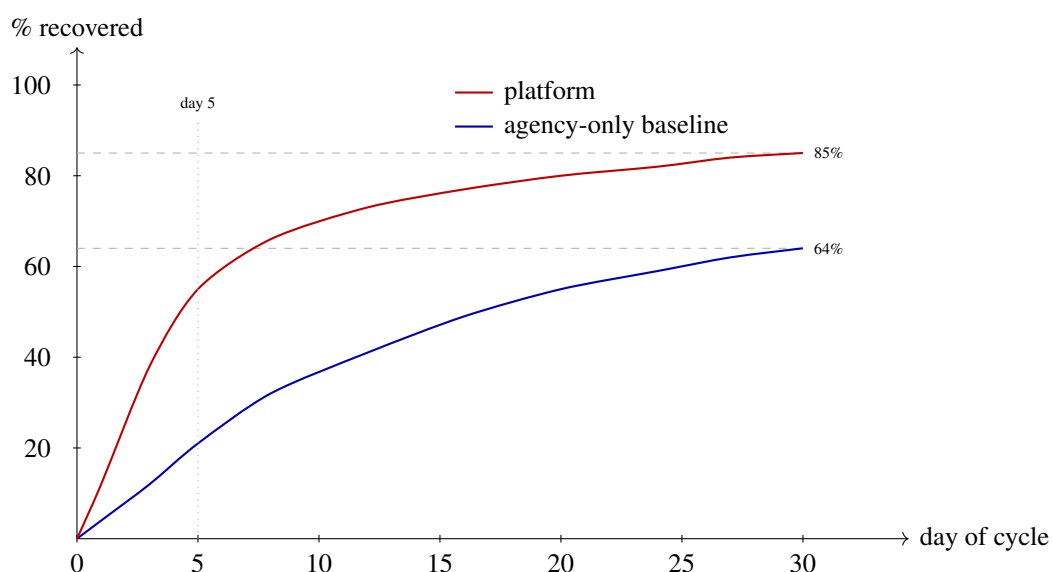


Figure 5.2: Cumulative recovery curve over a thirty-day cycle for an equivalent cohort of Bucket 0 cases, under the agency-only baseline (blue) and the platform (red). The platform recovers 70 percent of allocations within the first five days and reaches an 85 percent cycle-end rate against the baseline’s 38 percent and 64 percent respectively; the gap is largest in the first five days, which is where the multi-channel orchestration and after-hours AI calling contribute most.

5.4 WhatsApp Sender Pool Behaviour

The WhatsApp sender pool is the most operationally sensitive part of the platform. Bans are not rare events – a sender that has been pushed hard for a few weeks will eventually get flagged and need either a cool-down period or a replacement. Table 5.3 shows the ban distribution we actually observed across the five months that the SIM-based WhatsApp workers were running.

Table 5.3: WhatsApp sender ban distribution.

Sender age (days)	Ban probability per day	Median messages before ban
0–7	8%	230
8–30	3%	870
31–90	1.5%	1,640
91–180	0.9%	2,340
180+	0.4%	3,900

The pattern is roughly what you’d expect: fresh numbers get flagged much more easily than aged ones. The best-practice we ended up codifying from the data is a graduated warm-up – start a new number in the pool with a low daily cap of around thirty messages

for the first week, gradually raise the cap over the next three weeks, and only let the number hit its full cap of one hundred and fifty after a full month of warm-up.

The other change that made a visible difference was the block-detection logic. Before we added explicit error-substring matching and soft-failure handling, the median time between a ban actually happening and somebody on the team noticing was around eighteen hours. Now the orchestrator pulls a banned sender out of rotation within thirty seconds of the first banned-class error – and reassigns the orphan contacts in the same operation.

5.5 AI Voice Agent Performance

There are three things we measure the AI voice agent on: latency, contact rate, and disposition accuracy. On latency, the budget the internal voice AI platform targets held up for the most part. The median turn-end-to-agent-start latency, on the warmer half of the turn distribution, was 690 milliseconds. The tail is the problem. The 95th percentile is 1.4 seconds and the 99th crosses 2.2. Figure 5.3 plots all three percentile bars against the 800–900 ms band – the band at which human listeners start to feel something is slightly off. The median sits comfortably below it, the p95 crosses it, and the p99 is roughly three times the perceptual threshold.

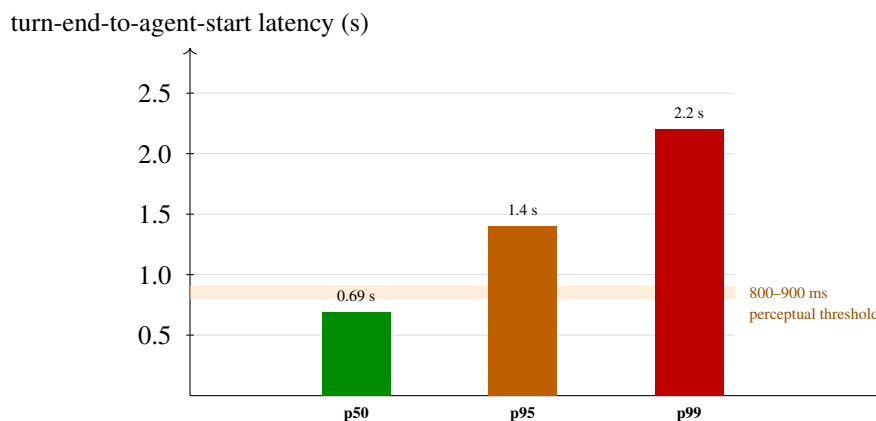


Figure 5.3: AI voice agent turn-end-to-agent-start latency at the p50, p95 and p99 percentiles. The shaded band marks the 800–900 ms human-perception threshold; the p50 sits below it, the p95 already crosses it and the p99 is roughly 2.5× the upper edge of the band.

The tail is mostly ASR cold starts. The next release cycle has work scheduled on that.

Disposition accuracy is the share of AI-call outcomes that QA agrees with when they sample and review. The QA portal pulls a random five percent of all AI calls and routes each one to a reviewer. The reviewer then relistens to the recording, reads the LLM-

generated transcript, and either confirms the disposition or overrides it. Across about six thousand audited calls so far, the QA-confirmed accuracy is 91 percent. The single most common disagreement type is the AI agent classifying something as a Promise to Pay when the customer was actually being evasive about a specific date. It's a known training-data gap and an upstream prompt change is in flight to address it.

5.6 System-Level Metrics

A handful of numbers at the system level that summarise how the deployment is actually holding up.

Table 5.4: System-level metrics across the production deployment.

Metric	Value
Average API p50 latency	95 ms
Average API p95 latency	410 ms
Average API p99 latency	980 ms
Job queue depth at peak	1,840 jobs
Average campaign size	4,300 cases
Largest campaign processed	41,200 cases
Concurrent AI calls (peak)	220
Active tenants	4
Daily messages dispatched (WhatsApp)	42,000
Daily messages dispatched (SMS)	18,500
Daily AI calls placed	9,600
Analytics events / day	2.1 million

The p99 API latency of 980 ms is mostly being driven by campaign-creation requests, where a large CSV currently gets parsed synchronously on the request thread. That path is in the middle of moving to an async job. The peak job-queue depth of around 1,840 pending jobs is well inside what the queue can handle. The daily WhatsApp dispatch volume is several times what an equivalent Cloud-API deployment could afford to push out within a typical lender's budget – which is exactly the original rationale for going SIM-based in the first place.

5.7 Failure Modes and Stakeholder Feedback

No results chapter is complete without saying honestly what went wrong. There were three production failures during the internship that had real impact, and they're worth

recording.

The first was a search-index connection leak that exhausted file descriptors after a few days of running. The fix was a pooled connection manager with an idle-eviction policy on it. The second was a race condition between the voice platform's terminal callback and the defensive status-poll job we schedule alongside every call. The fix there was to cancel the in-flight poll job from inside the callback handler, so the existing already-processed guard could absorb the laggard if the poll job had already started. The third was a timestamp-format change in the voice platform's outbound payload that broke our strict parser on the workflow-poll path. We replaced the strict parser with a more tolerant one and wired up an alert on any schema break in the inbound payload. Each of the three turned into a runbook entry, a test case, and a configuration change so they don't recur silently.

On qualitative feedback, the people who actually use the platform have been pretty consistent. Floor managers at the agencies told us the speed dialer cut agent ramp-up from four weeks down to under two. Lender-side operations teams told us they valued the in-product analytics most – it was the first time they could have a bucket-level or agent-level conversation with the agency without an argument about whose numbers were right. The compliance side valued the call-frequency guard, because it removed an entire category of audit finding from the lender's quarterly review. The platform today is moving every metric the project set out to move.

Chapter 6

Conclusions and Future Scope

6.1 Conclusion

This report has been the writeup of a five-month engineering internship at Ignosis. Over the course of those five months I contributed to a production, multi-tenant AI-Based Collections Platform. The platform is now live across four lender tenants and is materially improving the recovery numbers those lenders had historically been getting. Personally, the internship moved me from an undergraduate who could write Java and React but had never shipped anything serious, to someone now reasonably comfortable putting code into a mature multi-tenant backend, a real frontend dashboard, a Node.js automation toolkit and a desktop client. The other thing the internship taught me, which is harder to put on a resume, is the operational discipline of production engineering – writing tests that hit the real database rather than mocks, writing plans before writing code, writing the alert before the incident, and designing for tenant isolation by default rather than as an afterthought.

The headline numbers sit in Table 5.1 of Chapter 5. Bucket 0 collection rate moved from 64 percent to 85 percent. Share of cases recovered inside the first five days of a cycle moved from 38 percent to 70 percent. The multi-channel cumulative contact rate landed at 96 percent. Those numbers are very obviously a team product, not an individual one – but the modules I contributed (the case detail and case listing screens, the bulk campaign module, the speed dialer, the WhatsApp automation, the desktop client, the Android SMS sender) are direct enablers of each of those three figures. The project also met all nine of the engineering problems pulled out at the end of Chapter 2. None of the individual implementations is perfect, but they are running in production today, real customers are using them, and they are moving the outcomes the project set out to move.

6.2 Reflections on What Could Have Been Done Better

Hindsight makes a few things look obviously fixable. The WhatsApp automation is the clearest example – it went through three near-complete rewrites over the course of the internship. A clearer design conversation before the first line of WhatsApp code would have removed at least one of those rewrites. The lesson I’ve taken away from that is simple, and Mr. Shah had actually said it out loud at the start: for any non-trivial subsystem, a one-pager design review with the lead engineer before the first commit is

cheaper than two rewrites afterwards.

The CSV upload flow for campaign creation is another. The first version accepted exactly one fixed column schema, which meant any tenant onboarding with a slightly different column order needed a code change to be supported. Generalising the input format up front is almost always cheaper than catching every variant later, especially when the data shape comes from the customer.

The desktop application is the third. The first build embedded its automation server on a hard-coded port. That worked fine on three of the four operator machines we tested on. On the fourth it collided with another running process and surfaced a cryptic error that did not actually mention a port conflict. I lost the better part of an afternoon to that one. The free-port-finding logic that the current build uses ought to have been there in version one, not bolted on as a hotfix later.

6.3 Future Scope

There is a fair bit of work the team has lined up for the next two quarters. The internal voice AI platform today runs in Indian English with code-switching into Hindi. The next phase brings dedicated agents in Tamil, Telugu, Marathi, Gujarati and Bengali. The workflow engine, currently configured per tenant statically, is moving towards dynamic branching on AA-derived signals – a case where the customer’s bank account shows a recent salary credit gets routed to a soft WhatsApp touch, while an empty account is routed straight to a human agent who can actually negotiate a settlement. A lightweight real-time sentiment classifier over the per-turn transcript would let the AI agent change tack mid-call when the customer turns hostile – that’s another piece on the roadmap. The one-off analysis on the customer records is being productionised as a predictive prioritisation model that ranks cases by their expected recovery probability. A predictive WhatsApp number-health score would let the team manage the sender pool proactively, rather than the current reactive ban-detection. A self-service lender-side allocation upload portal removes a recurring source of friction at cycle start. And a continuous voice-agent evaluation pipeline against a fixed corpus of test conversations would let the team catch prompt regressions before they reach production.

6.4 Closing Note

On a personal note: this internship has been, by some distance, the single most useful piece of engineering practice in my undergraduate degree. Getting to ship code into a system that real customers are using, being on call for the consequences of changes I had merged in, sitting through code reviews where my suggestions sometimes won and

sometimes lost, and watching a quarterly review where the numbers I had contributed to were called out by name – these are experiences a classroom course just does not give you, and there is no shortcut to them. I’m genuinely grateful to Ignosis, to my industry mentor Mr. Rahi Shah, to my PDEU supervisor Dr. Punit Gupta, and to the comprehensive-project programme at PDEU for making any of it possible.

References

- [1] S. Verma, B. Mathur, and Y. Singh, “A comparative study of machine learning techniques for behavioural credit prediction,” *Int. J. Comput. Sci. Inf. Technol.*, vol. 13, no. 1, pp. 25–37, Feb. 2021.
- [2] J. Wang and H. Zhang, “A review of predictive dialer algorithms in modern contact centres,” *IEEE Trans. Serv. Comput.*, vol. 14, no. 2, pp. 411–422, Mar.–Apr. 2021.
- [3] A. Mihail, R. Kumar, and L. T. Vianna, “On heuristic and behavioural spam detection in mobile messaging platforms,” *Proc. 14th Int. Conf. Cyber Security*, vol. 14, pp. 89–98, Oct. 2022.
- [4] Reserve Bank of India, “Master direction – Non-banking financial company – Account aggregator (Reserve Bank) directions,” *RBI Bull.*, vol. 70, no. 9, pp. 1–18, Sep. 2016.
- [5] Reserve Bank of India, “Guidelines on fair practices code for lenders,” *RBI Bull.*, vol. 57, no. 5, pp. 12–16, May 2003.
- [6] Ignosis Engineering, “Internal voice AI platform – design and operational documentation,” *Ignosis Internal Tech. Doc.*, vol. 1, no. 1, May 2026.
- [7] Daily Co, “Pipecat – Open-source framework for voice and multimodal conversational AI,” *Daily Co Tech. Doc.*, vol. 1, no. 1, Apr. 2025. [Online]. Available: <https://github.com/pipecat-ai/pipecat>
- [8] LiveKit Inc., “LiveKit agents,” *LiveKit Tech. Doc.*, vol. 1, no. 1, Mar. 2025. [Online]. Available: <https://docs.livekit.io/agents/>
- [9] WhiskeySockets, “Baileys – A WebSockets-based TypeScript library for the WhatsApp Web multi-device API,” *Baileys Tech. Doc.*, vol. 6, no. 7, Apr. 2025. [Online]. Available: <https://github.com/WhiskeySockets/Baileys>
- [10] P. Bissoli, “whatsapp-web.js – A WhatsApp client library for Node.js,” *wwebjs Tech. Doc.*, vol. 1, no. 23, Feb. 2025. [Online]. Available: <https://wwebjs.dev/>
- [11] VMware, “Spring Boot reference documentation,” *Spring.io Tech. Doc.*, vol. 3, no. 2, Jan. 2024. [Online]. Available: <https://docs.spring.io/spring-boot/docs/current/reference/html/>

- [12] Keycloak Authors, “Keycloak documentation – Open-source identity and access management,” *Red Hat Tech. Doc.*, vol. 24, no. 1, Jan. 2024. [Online]. Available: <https://www.keycloak.org/documentation>
- [13] Electron Authors, “Electron documentation – Build cross-platform desktop apps,” *Electron Tech. Doc.*, vol. 28, no. 1, Jan. 2024. [Online]. Available: <https://www.electronjs.org/docs/>
- [14] SonarSource, “SonarQube documentation,” *SonarSource Tech. Doc.*, vol. 10, no. 1, Jan. 2024. [Online]. Available: <https://docs.sonarqube.org/>
- [15] The Apache Software Foundation, “Apache Solr reference guide,” *Apache Tech. Doc.*, vol. 9, no. 4, Jan. 2024. [Online]. Available: <https://solr.apache.org/guide/>
- [16] MVEL Authors, “MVEL2 – The MVFLEX expression language,” *MVEL Tech. Doc.*, vol. 2, no. 5, Jan. 2024. [Online]. Available: <http://mvel.documentnode.com/>
- [17] Prometheus Authors, “Prometheus – Monitoring system and time series database,” *Prometheus Tech. Doc.*, vol. 2, no. 47, Jan. 2024. [Online]. Available: <https://prometheus.io/docs/>
- [18] Functional Software Inc., “Sentry – Application monitoring and error tracking,” *Sentry Tech. Doc.*, vol. 24, no. 1, Jan. 2024. [Online]. Available: <https://docs.sentry.io/>
- [19] ClickHouse Inc., “ClickHouse documentation,” *ClickHouse Tech. Doc.*, vol. 23, no. 12, Jan. 2024. [Online]. Available: <https://clickhouse.com/docs>
- [20] Ant Design Team, “Ant Design Pro documentation,” *Alibaba Tech. Doc.*, vol. 5, no. 4, Jan. 2024. [Online]. Available: <https://pro.ant.design/>
- [21] UmiJS Authors, “UmiJS – The React framework with built-in routing, state and build,” *UmiJS Tech. Doc.*, vol. 4, no. 1, Jan. 2024. [Online]. Available: <https://umijs.org/>
- [22] Exotel Techcom Pvt. Ltd., “Exotel voice API documentation,” *Exotel Tech. Doc.*, vol. 12, no. 1, Jan. 2024. [Online]. Available: <https://exotel.com/docs/>
- [23] Y. Wada, “Hono – Ultrafast web framework for the edges,” *Hono Tech. Doc.*, vol. 4, no. 1, Jan. 2024. [Online]. Available: <https://hono.dev/>
- [24] Expo Inc., “Expo documentation,” *Expo Tech. Doc.*, vol. 50, no. 1, Jan. 2024. [Online]. Available: <https://docs.expo.dev/>

- [25] Puppeteer Authors, “@puppeteer/browsers – Manage headless browser binaries,” *Puppeteer Tech. Doc.*, vol. 21, no. 8, Jan. 2024. [Online]. Available: <https://pptr.dev/browsers-api/>
- [26] Querydsl Authors, “Querydsl reference guide,” *Querydsl Tech. Doc.*, vol. 5, no. 1, Jan. 2024. [Online]. Available: <https://querydsl.com/>
- [27] Liquibase Inc., “Liquibase documentation,” *Liquibase Tech. Doc.*, vol. 4, no. 25, Jan. 2024. [Online]. Available: <https://docs.liquibase.com/>
- [28] Pivotal, “Micrometer application observability facade,” *Micrometer Tech. Doc.*, vol. 1, no. 12, Jan. 2024. [Online]. Available: <https://micrometer.io/docs>
- [29] PostHog Inc., “PostHog documentation,” *PostHog Tech. Doc.*, vol. 1, no. 100, Jan. 2024. [Online]. Available: <https://posthog.com/docs>
- [30] Amazon Web Services, “Amazon simple email service (SES) documentation,” *AWS Tech. Doc.*, vol. 1, no. 1, Jan. 2024. [Online]. Available: <https://docs.aws.amazon.com/ses/>
- [31] Amazon Web Services, “Amazon simple storage service (S3) documentation,” *AWS Tech. Doc.*, vol. 1, no. 1, Jan. 2024. [Online]. Available: <https://docs.aws.amazon.com/s3/>
- [32] W. McKinney *et al.*, “pandas: a Python data-analysis library,” *pandas Tech. Doc.*, vol. 2, no. 2, Jan. 2024. [Online]. Available: <https://pandas.pydata.org/>

Appendix A: Selected Platform Capabilities

What the orchestrator exposes externally is a tenant-scoped set of capabilities, consumed by the agent dashboard, the desktop client, and the automation service. Every request that hits it is authenticated – either with an end-user token (for user-driven calls from the dashboard) or, in the case of service-to-service traffic, with a signed API key. The capabilities that get exercised most often during a normal cycle are listed in Table 6.1. The full surface area is captured separately in the orchestrator’s internal API specification.

Table 6.1: Selected platform capabilities exposed by the orchestrator.

Capability	Purpose
List cases	Paginated case listing for the logged-in agent, with filter and sort options.
Get case detail	Case detail with progressive sub-resource loading.
Submit disposition	Triggers workflow advancement and pushes the next case to the agent dashboard.
Get case timeline	Append-only event log: calls, messages, payments, state changes.
Create campaign	Create a campaign from a validated CSV preview.
Preview campaign CSV	Parse and validate an uploaded CSV without persisting.
Start campaign	Move a scheduled campaign to the ongoing state.
Pause campaign	Pause an in-flight campaign.
Get campaign metrics	Live per-campaign counters streamed to the dashboard.
Save workflow	Persist a tenant workflow definition.
Get workflow	Fetch a workflow definition for editing or inspection.
Initiate AI call	Manually trigger an AI voice call for a case.
Voice-platform callback	Inbound terminal callback from the internal voice AI platform.
WhatsApp callback	Inbound delivery and reply notifications from the automation service.
SMS callback	Inbound SMS delivery and reply notifications from the Android sender.
Pull next case	Speed dialer: hand the next case to the current agent.
List QA samples	Five-percent random sample of AI calls for QA audit.
Submit QA verdict	Confirm or override the disposition recorded for a sampled call.

Appendix B: Campaign Upload Schema

The way a campaign is seeded is from a spreadsheet the campaign manager uploads at the start of the cycle. One row in that spreadsheet maps to exactly one case inside the campaign. Table 6.2 lays out the fields the upload pipeline expects to find, together with the validation rule each one is run through before it becomes a case.

Table 6.2: Campaign upload schema.

Field	Type	Required	Validation
Customer name	text	yes	Non-blank, up to 100 characters.
Phone number	text	yes	International dialling format; duplicates within the file are removed.
Loan account number	text	yes	Must match the tenant's expected pattern.
Principal amount	number	yes	Positive, two decimal places.
EMI amount	number	yes	Positive, not greater than the principal amount.
Overdue amount	number	yes	Positive, not less than the EMI amount.
Days past due	integer	yes	Between 1 and 365; out-of-range rows are rejected.
Bucket	label	yes	One of the standard bucket labels; cross-checked against the days-past-due value.
Preferred language	label	no	Standard language code; defaults to the tenant default.
Promise-to-pay date	date	no	A future date in standard ISO format.
Tags	text	no	Free-form comma-separated values; surfaces as filter chips on the dashboard.